



HTML5

CHEAT SHEET

2026 EDITION

Essential HTML5 Fundamentals for Students, Interviews & Developers

HTML WORKFLOW



1. Browser Requests HTML



2. HTML Parser Parses Markup



3. DOM (Document Object Model)



4. Render Tree (CSSOM + DOM)



5. Layout & Paint Rendered on Screen

BASIC HTML DOCUMENT STRUCTURE

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document Title</title>
    <meta name="description" content="Page description">
    <link rel="stylesheet" href="styles.css">
    <script src="script.js" defer></script>
  </head>
  <body>
    <header>...</header>
    <main>
      <h1>Hello, HTML5!</h1>
      <p>Welcome to the future of the web.</p>
    </main>
    <footer>...</footer>
  </body>
</html>
```

IMPORTANT TAGS

- html
- head
- body
- title
- meta
- link
- style
- script
- noscript
- base

HTML SYNTAX RULES

- ✓ HTML is not case sensitive but lowercase is recommended.
- ✓ All tags must be properly nested.
- ✓ Every start tag must have an end tag.
- ✓ Void elements do not have a closing tag.
`<img> <br> <hr> <input> <meta> <link>`
- ✓ Attribute values must be quoted.
- ✓ Use meaningful, semantic HTML.
- ✓ Validate your HTML for error-free code.

ELEMENTS vs TAGS

TAG: The markup used to define an element.

Example:

`<p> ... </p>`

ELEMENT: The content inside the tags.

Example:

`<p> Hello</p>`

ATTRIBUTES

Attributes provide additional information about elements and are always specified in the start tag.

Example:

```
<a href="https://example.com"
  target="_blank" title="Visit Website">Link</a>
```

GLOBAL ATTRIBUTES (Can be used on all HTML elements)



id

Unique identifier for an element

.class

class

Specifies one or more CSS classes



style

Inline CSS styles



title

Extra information (tooltip)



lang

Language of the element content



dir

Text direction (ltr or rtl)



hidden

Hides the element



tabindex

Defines tab order for focus



draggable

Makes element draggable



contenteditable

Makes content editable



spellcheck

Enables/Disables spell checking



translate

Should content be translated (yes/no)



CHARACTER ENCODING

Always use UTF-8 encoding for maximum compatibility.

```
<meta charset="UTF-8">
```

Benefits:

- ✓ Supports all languages
- ✓ Compact & efficient
- ✓ Recommended by HTML Living Standard



COMMENTS

Comments are not rendered in the browser.

Syntax:

```
<!-- This is a comment
      It will not appear in
      the browser -->
```

Use comments to explain your code.



NESTING RULES

- ✓ Elements must be properly nested.
- ✓ Never overlap elements.
- ✓ A parent element must contain its child elements.

Correct:

```
<div>
  <p>Text</p>
</div>
```

Incorrect:

```
<div>
  <p>Text</div></p>
```



WHITESPACE

HTML collapses multiple whitespaces into one.

The following are treated the same:

- Spaces
- Tabs
- Newlines

To preserve whitespace, use `<pre>` or CSS.

```
<pre> Preformatted Text </pre>
```

QUICK REFERENCE



DOCTYPE

Defines document type & HTML version.

```
<!DOCTYPE html>
```



LANGUAGE

Set document language using lang attribute.

```
<html lang="en">
```



VIEWPORT

Make your site responsive.

```
<meta name="viewport"
  content="width=device-width,
  initial-scale=1.0">
```



EXTERNAL CSS

Link external stylesheet.

```
<link rel="stylesheet"
  href="styles.css">
```



EXTERNAL JS

Link external JavaScript.

```
<script src="script.js"
  defer></script>
```



NOSCRIPT

Content for users with JavaScript disabled.

```
<noscript>
  JavaScript is disabled.
</noscript>
```



BASE URL

Define base URL for relative links.

```
<base href="https://
  example.com/">
```



KEY TAKEAWAYS

HTML provides the structure of the web. Master the fundamentals to build semantic, accessible & future-proof web applications.



Write semantic HTML today for a better web tomorrow.

– HTML Living Standard (2026)

VOID ELEMENTS (No Closing Tag)

```
<img> <br> <hr> <input> <meta>
<link> <source> <track> <area> <base>
```




TEXT ELEMENTS

TAG	DESCRIPTION	EXAMPLE
<h1> - <h6>	Headings (level 1 to 6)	Main Title
<p>	Paragraph	This is a paragraph.
	Inline container	Inline text
<div>	Block-level container	Block content
<pre>	Preformatted text	<pre>Code here</pre>
<code>	Inline code	Use <code>code()</code>
<kbd>	Keyboard input	Press <kbd>Ctrl</kbd> + C
<samp>	Sample output	<samp>Success!</samp>
<var>	Variable	The value of <var></var>
<blockquote>	Block quotation	<blockquote>Quote...</blockquote>
<q>	Inline quotation	He said, <q>Hello!</q>
<cite>	Citation	— <cite>HTML Spec</cite>
<abbr>	Abbreviation	<abbr title="HyperText Markup Language">HTML</abbr>
<address>	Contact info	<address>123 Web St...</address>
<time>	Date / Time	<time datetime="2026-05-01">May 1, 2026</time>
<mark>	Highlighted text	This is <mark>important</mark>.
	Strong importance	Important!
	Emphasis	Emphasized text
<small>	Side note / fine print	Terms apply
<sub>	Subscript	H₂O
<sup>	Superscript	x²
	Bold (no extra importance)	Bold text
<i>	Italic (no extra importance)	<i>Italic text</i>
<u>	Underline	<u>Underlined text</u>
<s>	Strikethrough (no longer accurate)	<s>Old price</s>
	Deleted text	Was \$20
<ins>	Inserted text	<ins>Now \$15</ins>



LISTS

TAG	DESCRIPTION	EXAMPLE
	Unordered list	Item 1
	Ordered list	Item 1
	List item	List item
<dl>	Description list	<dl>...</dl>
<dt>	Term in description list	<dt>HTML</dt>
<dd>	Description in description list	<dd>Markup language</dd>



LINKS & ANCHORS

ATTRIBUTE / TAG	DESCRIPTION	EXAMPLE
<a>	Anchor / Hyperlink	Link
href	Link destination	href="/about.html"
target	Where to open link	target="_blank"
download	Download the resource	Download
mailto:	Email link	Email
tel:	Phone link	Call
rel	Relationship between links	rel="noopener noreferrer"
Fragment	Link to an element on page	Go



IMAGES

TAG / ATTRIBUTE	DESCRIPTION	EXAMPLE
	Embed an image	
<picture>	Responsive image container	<picture>...</picture>
<source>	Image source for <picture>	<source srcset="img.avif" type="image/avif">
srcset	Multiple image resolutions	srcset="img-400.jpg 400w, img-800.jpg 800w"
sizes	Define image display sizes	sizes="(max-width: 600px) 100vw, 600px"
loading	Lazy load images	loading="lazy"
decoding	Decoding hint	decoding="async"
alt	Alternative text (accessibility)	alt="A beautiful landscape"



MEDIA (AUDIO & VIDEO)

TAG / ATTRIBUTE	DESCRIPTION	EXAMPLE
<audio>	Embed audio content	<audio controls src="song.mp3"></audio>
<video>	Embed video content	<video controls src="video.mp4"></video>
<source>	Media source	<source src="movie.webm" type="video/webm">
<track>	Text tracks (captions, subs)	<track kind="subtitles" src="sub.vtt" srclang="en" label="English" default>



OTHER USEFUL ELEMENTS

TAG	DESCRIPTION	EXAMPLE
 	Line break	First line Second line
<hr>	Thematic break (horizontal line)	<hr>
<wbr>	Word break opportunity	This<wbr>Is<wbr>Long
<ruby>	Ruby annotation (East Asian)	<ruby>漢<rt>kan</rt></ruby>
<bdi>	Bidirectional isolation	<bdi>مما</bdi>
<bdo>	Bidirectional override	<bdo dir="rtl">abc</bdo>



QUICK REFERENCE SUMMARY



TEXT

- Headings: <h1> - <h6>
- Paragraph: <p>
- Inline: , ,
- Code: <code>, <pre>, <kbd>
- Quotes: <blockquote>, <q>
- Others: <abbr>, <cite>, <time>



LISTS

- Unordered:
- Ordered:
- List Item:
- Description: <dl>
- Term: <dt>
- Description: <dd>



LINKS

- Link: <a>
- Attributes: href, target, download, rel
- Email: mailto:
- Phone: tel:
- Fragment: #id



IMAGES

- Image:
- Responsive: <picture>
- Sources: <source>
- srcset, sizes
- Lazy load: loading="lazy"
- Alt text: alt



MEDIA

- Audio: <audio>
- Video: <video>
- Source: <source>
- Captions: <track>



These elements are the building blocks of modern web development.
Master them to build accessible, semantic and future-ready websites.





TABLES REFERENCE		
TAG	DESCRIPTION	EXAMPLE
<table>	Table container	<table>...</table>
<caption>	Table title / caption	<caption>Data Table</caption>
<thead>	Header section (top)	<thead>...</thead>
<tbody>	Body section (rows)	<tbody>...</tbody>
<tfoot>	Footer section (bottom)	<tfoot>...</tfoot>
<tr>	Table row	<tr>...</tr>
<th>	Table header cell	<th scope="col">Name</th>
<td>	Table data cell	<td>John</td>
<colgroup>	Group of columns	<colgroup>...</colgroup>
<col>	Column definition	<col span="2" style="width:100px">
rowspan	Row span (attribute)	<td rowspan="2">Cell</td>
colspan	Column span (attribute)	<td colspan="3">Data</td>

TABLE EXAMPLE

Name	Age	Role
Alice	24	Developer
Bob	26	Designer
Charlie	28	Manager

```
<table>
  <caption>Team Members</caption>
  <thead>
    <tr><th>Name</th><th>Age</th><th>Role</th></tr>
  </thead>
  <tbody>
    <tr><td>Alice</td><td>24</td><td>Developer</td></tr>
    <tr><td>Bob</td><td>26</td><td>Designer</td></tr>
    <tr><td>Charlie</td><td>28</td><td>Manager</td></tr>
  </tbody>
</table>
```

FORMS REFERENCE		
ELEMENT	DESCRIPTION	EXAMPLE
<form>	Form container	<form action="/submit" method="post">
<label>	Label for form control	<label for="name">Name</label>
<input>	Input control (various types)	<input type="text" id="name">
<textarea>	Multi-line text input	<textarea rows="4"></textarea>
<select>	Dropdown list	<select><option>One</option></select>
<option>	Option in select	<option value="1">One</option>
<button>	Button (submit/reset/button)	<button type="submit">Send</button>
<fieldset>	Group related controls	<fieldset>...</fieldset>
<legend>	Caption for fieldset	<legend>Personal Info</legend>
<datalist>	Predefined options for input	<input list="browsers">
<meter>	Scalar measurement	<meter min="0" max="100" value="60">
<progress>	Task progress indicator	<progress value="70" max="100">
<output>	Calculation result	<output for="a b" name="result">
<input type="file">	File upload control	<input type="file" accept="image/*">

SEMANTIC HTML LAYOUT

Semantic elements clearly describe the purpose of different parts of a web page.

<header>
Introductory content (logo, title, etc.)

<nav>
Navigation links

<main>
Main content of the document

<aside>
Related content (sidebar)

<section> / <article>
Independent content block

<footer>
Footer information (copyright, links, etc.)

OTHER SEMANTIC ELEMENTS			
<section>	Thematic grouping of content	<summary>	Visible heading for <details>
<article>	Self-contained independent content	<search>	Represents search functionality
<details>	Hide / show additional info	<figure>	Self-contained content (image, etc.)
<dialog>	Modal or non-modal dialog box	<figcaption>	Caption for <figure>

ALL <INPUT> TYPES

text

password

email

tel

url

search

number

range

date

time

datetime-local

month

week

file

checkbox

radio

color

hidden

submit

reset

button

image

FORM VALIDATION & ATTRIBUTE REFERENCE		
ATTRIBUTE	DESCRIPTION	EXAMPLE
required	Field must be filled	<input required>
pattern	Regex pattern validation	<input pattern="[A-Za-z]+">
maxlength	Max number of characters	<input maxlength="20">
minlength	Min number of characters	<input minlength="3">
min	Minimum value	<input type="number" min="0">
max	Maximum value	<input type="number" max="100">
placeholder	Hint text inside field	<input placeholder="Your name">
readonly	Cannot be edited	<input readonly>
disabled	Control is disabled	<input disabled>
autocomplete	Enable autocomplete	<input autocomplete="name">
autofocus	Automatically focused	<input autofocus>
multiple	Allow multiple values	<input type="file" multiple>
accept	Accepted file types (file input)	<input type="file" accept=".png,.jpg">

WHY SEMANTIC HTML?

Improves accessibility for assistive technologies

Better SEO and discoverability

Cleaner code & easier maintenance

Enhanced user experience

Future-proof and standard-compliant



ACCESSIBILITY (A11Y)

Build inclusive websites for everyone.

	Semantic HTML	Use correct HTML elements <header>, <nav>, <main>, <button> etc.
	ARIA Basics	Use ARIA roles, states and properties when semantic HTML is not enough. Example: role="button" aria-label="Close"
	Labels	Always use <label> for form controls. Use aria-label for non-visible labels.
	Keyboard Navigation	Ensure all interactive elements are focusable and operable via keyboard.
	Focus Order	Logical tab order. Use tabindex="0" sparingly. Avoid tabindex > 0.
	Screen Readers	Test with NVDA, JAWS, VoiceOver. Provide meaningful text alternatives.
	Landmarks	Use landmark roles: <header>, <nav>, <main>, <aside>, <footer>.
	Skip Links	Provide skip navigation to main content. Example: Skip to content
	Alt Text	Provide meaningful alt for images. alt="Describe the image"
	Color Contrast	Maintain sufficient contrast ratio. Normal text: 4.5:1 Large text: 3:1



SEO (SEARCH ENGINE OPTIMIZATION)

Help search engines understand your content.

	Title	<title>Page Title</title> Keep it concise and descriptive.
	Meta Description	<meta name="description" content="..."> Summarize page content (150–160 chars).
	Canonical URL	<link rel="canonical" href="https://example.com/page"> Prevent duplicate content issues.
	Open Graph	<meta property="og:title" content="Title"> Improve sharing on social platforms.
	Twitter Cards	<meta name="twitter:card" content="summary_large_image"> Optimize for Twitter sharing.
	Structured Data	Use JSON-LD for rich results. Example: Schema.org markup
	Headings	Use proper heading hierarchy (H1 → H2 → H3). Don't skip levels.
	Image SEO	Use descriptive file names, alt text, and appropriate image format.



PERFORMANCE OPTIMIZATION

Make your website fast and efficient.

	Lazy Loading	 Load images and iframes only when needed.
	Preload	<link rel="preload" href="style.css" as="style"> Preload critical resources.
	Prefetch	<link rel="prefetch" href="next-page.html"> Prefetch resources for future navigation.
	Preconnect	<link rel="preconnect" href="https://fonts.googleapis.com"> Establish early connections to origins.
	DNS Prefetch	<link rel="dns-prefetch" href="//example.com"> Resolve DNS early.
	Image Optimization	Use modern formats (WebP, AVIF), compress and set correct dimensions.
	Critical Rendering	Inline critical CSS. Defer non-critical CSS & JS.
	Resource Hints	Use resource hints (preload, prefetch, preconnect) to speed up loading.



SECURITY BEST PRACTICES

Protect users and your website.

	XSS Protection	Sanitize user input and output. Escape HTML, use frameworks.
	Content Security Policy	<meta http-equiv="Content-Security-Policy" content="default-src 'self'"> Mitigate XSS and injection attacks.
	HTTPS	Always use HTTPS for secure communication.
	noopener	 Prevent access to window.opener.
	noreferrer	 Prevent referrer leakage.
	iframe sandbox	<iframe sandbox="allow-scripts allow-same-origin"> Restrict iframe capabilities.
	Subresource Integrity	<script src="app.js" integrity="sha384-..." crossorigin="anonymous"></script> Ensure resources are not tampered.



QUICK CHECKLIST



ACCESSIBILITY

- ✓ Semantic HTML used
- ✓ Images have alt text
- ✓ Form labels are defined
- ✓ Keyboard navigable
- ✓ Good color contrast
- ✓ Skip link available
- ✓ Tested with screen reader



SEO

- ✓ Unique & descriptive title
- ✓ Meta description added
- ✓ Canonical URL set
- ✓ Open Graph tags added
- ✓ Headings in proper order
- ✓ Structured data added
- ✓ Images optimized for SEO



PERFORMANCE

- ✓ Lazy loading enabled
- ✓ Critical CSS inlined
- ✓ Images optimized
- ✓ Preload important resources
- ✓ Minified CSS & JS
- ✓ Used resource hints
- ✓ Tested with Lighthouse



SECURITY

- ✓ HTTPS enabled
- ✓ CSP implemented
- ✓ User input sanitized
- ✓ Use rel="noopener noreferrer"
- ✓ SRI for external resources
- ✓ Iframe sandboxed
- ✓ Dependencies updated



TOOLS TO USE

- Google Lighthouse
- WAVE Accessibility
- PageSpeed Insights
- GTmetrix
- Security Headers
- Schema Validator
- Web.dev



Good HTML is more than code.
It's about accessibility, performance, SEO and security.



Write better HTML.
Build better web.





HTML5 WEB APIS

API	DESCRIPTION & KEY METHODS
Geolocation API	Get user's location information. Methods: <code>getCurrentPosition()</code> , <code>watchPosition()</code>
Clipboard API	Read & write clipboard content. Methods: <code>navigator.clipboard.readText()</code> , <code>writeText()</code>
Drag & Drop API	Make elements draggable. Events: <code>dragstart</code> , <code>dragover</code> , <code>drop</code> , <code>dragend</code>
History API	Navigate session history programmatically. Methods: <code>pushState()</code> , <code>replaceState()</code> , <code>go()</code>
Web Storage	Store key/value data in the browser. Methods: <code>localStorage</code> , <code>sessionStorage</code>
Fullscreen API	Display elements in fullscreen mode. Methods: <code>requestFullscreen()</code> , <code>exitFullscreen()</code>
File API	Read & manipulate files. Interfaces: <code>File</code> , <code>FileReader</code> , <code>Blob</code>
Web Share API	Share text, links, files from web apps. Method: <code>navigator.share()</code>
Web Speech API	Speech synthesis & recognition. Interfaces: <code>SpeechSynthesis</code> , <code>SpeechRecognition</code>
Canvas API	Draw graphics via JavaScript. Methods: <code>getContext('2d')</code> , <code>drawImage()</code> , <code>fillRect()</code>
SVG	Scalable Vector Graphics for resolution-independent images. Elements: <code><svg></code> , <code><path></code> , <code><circle></code> , <code><rect></code>
Web Components	Create reusable custom elements. Technologies: Custom Elements, Shadow DOM, HTML Templates

DEVELOPER TOOLS (BUILT-IN)

TOOL	USE CASE
Elements Panel	Inspect & edit HTML/CSS in real time.
DOM Tree	View document structure & relationships.
Accessibility Inspector	Check accessibility issues & best practices.
Network Tab	Monitor requests, headers, status, and timing.
Performance Tab	Record, analyze, and optimize performance.
Lighthouse	Audit performance, SEO, accessibility, and best practices.
Device Toolbar	Test responsive layouts on different devices.

COMPLETE HTML5 TAG REFERENCE (QUICK VIEW)

Document & Metadata

`<!DOCTYPE>` `<html>` `<head>`
`<title>` `<meta>` `<link>` `<style>`
`<script>` `<noscript>` `<base>`

Sectioning

`<body>` `<header>` `<nav>` `<main>`
`<section>` `<article>` `<aside>`
`<footer>` `<address>`

Content

`<h1>` - `<h6>` `<p>` `<hr>` `<br>`
`<div>` `<span>` `<pre>` `<blockquote>`
`<q>` `<cite>` `<code>` `<kbd>` `<samp>`
`<var>` `<mark>` `<strong>` `<em>` `<small>`
`<sub>` `<sup>` `<ins>` `<del>` `<u>` `<s>`
`<b>` `<i>` `<time>` `<abbr>`

Grouping

`<ul>` `<ol>` `<li>` `<dl>` `<dt>` `<dd>`
`<figure>` `<figcaption>` `<details>` `<summary>`
`<dialog>` `<search>`

Tables

`<table>` `<caption>` `<thead>` `<tbody>`
`<tfoot>` `<tr>` `<th>` `<td>`
`<colgroup>` `<col>`

Forms

`<form>` `<label>` `<input>` `<textarea>`
`<select>` `<option>` `<button>` `<fieldset>`
`<legend>` `<datalist>` `<meter>` `<progress>`
`<output>`

Media

`<img>` `<picture>` `<source>`
`<audio>` `<video>` `<track>`

Embedded Content

`<embed>` `<object>` `<param>`
`<iframe>`

Interactive

`<a>` `<button>` `<summary>` `<details>`

TAG LEGEND

- Document Root
- Metadata
- Sectioning
- Content
- Forms
- Media
- Interactive
- Embedded

VOID ELEMENTS (No Closing Tag)

`<area>` `<base>` `<br>` `<col>` `<embed>`
`<hr>` `<img>` `<input>` `<link>` `<meta>`
`<param>` `<source>` `<track>` `<wbr>`

Void elements do not have content and do not require closing tags.



WEB COMPONENTS IN DETAIL

KEY BUILDING BLOCKS

TECHNOLOGY	PURPOSE
Custom Elements	Define new HTML tags. <code>class MyElement extends HTMLElement { ... }</code> <code>customElements.define('my-element', MyElement);</code>
Shadow DOM	Encapsulate styles & markup. <code>element.attachShadow({ mode: 'open' });</code>
HTML Template	Store HTML content for later use. <code><template id="tpl"><div>...</div></template></code>
Slot	Create insertion points in components. <code><slot></slot></code>

EXAMPLE: SIMPLE CUSTOM ELEMENT

```
class HelloBox extends HTMLElement {
  constructor() {
    super();
    const shadow = this.attachShadow({ mode: 'open' });
    shadow.innerHTML = `<style>p{color:#0ea5e9;font-weight:bold}</style>`
      + `<p><slot name="text">Hello!</slot></p>`;
  }
}
customElements.define('hello-box', HelloBox);
/* Usage: <hello-box><span slot="text">HTML5 Rocks!</span></hello-box> */
```



PWA (PROGRESSIVE WEB APP) BASICS

Web App Manifest	Provide app metadata for installability. Example: <code>manifest.json</code>
Theme Color	Customize browser UI color. Example: <code><meta name="theme-color" content="#2563eb"></code>
Icons	Use multiple icon sizes. Example: <code><link rel="icon" sizes="192x192" href="icon.png"></code>
Offline Support	Cache assets & enable offline usage. Use Service Workers.
Installability	Add to home screen & run like a native app. Requires HTTPS + valid manifest + service worker.



BEST PRACTICES CHECKLIST

CATEGORY	BEST PRACTICE
Structure	✓ Use proper heading hierarchy (h1 → h6).
Semantics	✓ Use semantic HTML elements for meaningful content.
Accessibility	✓ Ensure keyboard navigability & screen reader support.
Forms	✓ Use labels, placeholders, and validation attributes.
Images	✓ Provide meaningful alt text for all images.
Styling	✓ Use external CSS for maintainable & reusable styles.
Scripting	✓ Use external JS and defer/async for better performance.
Code Quality	✓ Write clean, well-formatted, and valid HTML.
Responsive	✓ Use responsive images (srcset, sizes) and media queries.
Performance	✓ Optimize assets, enable lazy loading, and minimize requests.
SEO	✓ Use meaningful titles, meta tags, and structured data.
Production	✓ Follow security best practices & test across browsers.



KEY TAKEAWAY

- ✓ HTML5 is the foundation of the modern web.
- ✓ Use semantic, accessible, and SEO-friendly markup.
- ✓ Optimize for performance and security.
- ✓ Leverage modern APIs and best practices.
- ✓ Build for all users, on all devices.

